

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Hafiz Perdana

NIM. 2410817210027

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Hafiz Perdana
NIM : 2410817210027

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Erza Raihan
NIM. 2310817210027

Irham Maulani Abdul Gani, S.Kom.,
M.Kom
NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
SOAL 1	6
A. Source Code.....	7
B. Output Program	33
C. Pembahasan	34
TAUTAN GIT	42

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Tampilan List Jawaban Soal 1	33
Gambar 2. Screenshot Hasil Tampilan Detail Jawaban Soal 1	33

DAFTAR TABEL

Tabel 1. Source Code MainActivity.kt Soal 1	7
Tabel 2. Source Code MyApplication.kt Soal 1	7
Tabel 3. Source Code AnimeData.kt Soal 1	8
Tabel 4. Source Code NetworkResult.kt Soal 1	10
Tabel 5 Source Code AppPreferencesRepository.kt Soal 1	11
Tabel 6. Source Code AnimeDao.kt Soal 1	11
Tabel 7. Source Code AnimeDatabase.kt Soal 1	12
Tabel 8. Source Code AnimeEntitiy.kt Soal 1	13
Tabel 9. Source Code NetworkModule.kt Soal 1	13
Tabel 10. Source Code TmdbApiService.kt.....	15
Tabel 11. Source Code TmdbResponse.kt Soal 1	15
Tabel 12. Source Code AnimeRepository.kt Soal 1	16
Tabel 13. Source Code Anime.kt Soal 1	20
Tabel 14. Source Code AppNavigation.kt Soal 1.....	20
Tabel 15. Source Code DetailScreen.kt Soal 1	21
Tabel 16. Source Code HomeScreen.kt Soal 1.....	23
Tabel 17. Source Code SettingScreen.kt Soal 1	28
Tabel 19. Source Code AnimeViewModel.kt Soal 1	30
Tabel 20. Source Code AnimeViewModelFactory.kt Soal 1	31

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- b. Gunakan KotlinX Serialization sebagai library JSON.
- c. Gunakan library seperti Coil atau Glide untuk image loading.
- d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
- f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
- g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus dapat mempertahankan fitur-fitur yang sudah dibuat pada modul sebelumnya.

A. Source Code

MainActivity.kt

Tabel 1. Source Code MainActivity.kt Soal 1

1	package com.example.modul5compose
2	
3	import android.os.Bundle
4	import androidx.activity.compose.setContent
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.compose.foundation.layout.fillMaxSize
7	import androidx.compose.material3.MaterialTheme
8	import androidx.compose.material3.Surface
9	import androidx.compose.material3.darkColorScheme
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.graphics.Color
12	import
	com.example.modul5compose.data.datastore.AppPreferencesRepository
13	import
	com.example.modul5compose.ui.navigation.AppNavigation
14	
15	class MainActivity : AppCompatActivity() {
16	override fun onCreate(savedInstanceState: Bundle?) {
17	super.onCreate(savedInstanceState)
18	AppPreferencesRepository.applySavedLanguage(this)
19	setContent {
20	MaterialTheme(colorScheme =
	darkColorScheme(background = Color.Black)) {
21	Surface(modifier = Modifier.fillMaxSize(),
	color = Color.Black) {
22	AppNavigation()
23	}
24	}
25	}
26	}
27	}

MyApplication.kt

Tabel 2. Source Code MyApplication.kt Soal 1

1	package com.example.modul5compose
2	
3	import android.app.Application
4	import android.content.pm.ApplicationInfo
5	import timber.log.Timber
6	

```

7 class MyApplication : Application() {
8     override fun onCreate() {
9         super.onCreate()
10
11         val isDebug = (applicationInfo.flags and
ApplicationInfo.FLAG_DEBUGGABLE) != 0
12
13         if (isDebug) {
14             Timber.plant(Timber.DebugTree())
15         }
16     }
17 }

```

Folder data: AnimeData.kt

Tabel 3. Source Code AnimeData.kt Soal 1

```

1 package com.example.modul5compose.data
2
3 import com.example.modul5compose.R
4 import com.example.modul5compose.model.Anime
5
6 object AnimeData {
7     val animeList = listOf(
8         Anime(
9             1,
10            "I Made Friends with the Second Prettiest Girl
in My Class",
11            "2026",
12            "01-01-2026",
13            "Maki Maehara, siswa SMA yang pemalu, semakin
dekat dengan Umi Asanagi berkat kesamaan minat mereka.
Seiring waktu, Umi jatuh cinta padanya, tetapi Maki ragu
menerima perasaannya karena kurang percaya diri.",
14            "Maki Maehara, a shy high school student, grows
close to the popular Umi Asanagi through their shared
interests. As their friendship deepens, Umi falls in love
with him, but Maki hesitates to accept her feelings due to
his lack of confidence.",
15            R.drawable.img_kuranika,
16            "https://myanimelist.net/anime/56734/Class_de_2-
banme_ni_Kawaii_Onnanoko_to_Tomodachi_ni_Natta",
17            null),
18            Anime(
19                2,
20                "Blue Box",

```


21	"2024",
22	"03-10-2024",
23	"Taiki Inomata, seorang pemain bulu tangkis, berjuang meraih tingkat nasional sambil berusaha mendekati Chinatsu Kano, gadis yang ia sukai dan yang kini tinggal serumah dengannya.",
24	"Taiki Inomata, a badminton player, is determined to reach the national stage while growing closer to his crush, Chinatsu Kano, a talented basketball player who unexpectedly ends up living under the same roof as him.",
25	R.drawable.img_aonohako,
26	"https://myanimelist.net/anime/57181/Ao_no_Hako",
27	null),
28	Anime(
29	3,
30	"I Want to End This Love Game",
31	"2026",
32	"01-01-2026",
33	"Sahabat masa kecil Yukiya Asagi dan Miku Sakura menjadikan ucapan "aku cinta kamu" sebagai permainan, tetapi seiring waktu, persaingan mereka berubah menjadi perasaan cinta yang sesungguhnya.",
34	"Childhood friends Yukiya Asagi and Miku Sakura turn saying "I love you" into a game, but as they grow older, their playful rivalry gradually develops into real romantic feelings.",
35	R.drawable.img_aishiterugame,
36	"https://myanimelist.net/anime/61839/Aishiteru_Game_wo_Owarasetai?q=aishiteru%20game%20&cat=anime",
37	null),
38	Anime(
39	4,
40	"Hori-san and Miyamura-kun",
41	"2021",
42	"09-01-2021",
43	"Meski memiliki kepribadian yang berbeda, Kyouko Hori dan Izumi Miyamura menjadi dekat setelah saling mengetahui sisi tersembunyi masing-masing, yang kemudian berkembang menjadi persahabatan dan kisah cinta yang hangat.",
44	"Despite their contrasting personalities, Kyouko Hori and Izumi Miyamura become close after discovering each other's hidden sides, leading to a heartwarming friendship and romance.",

45	R.drawable.img_horimiya,
46	"https://myanimelist.net/anime/42897/Horimiya?q=horimiya&cat=anime",
47	null),
48	Anime(
49	5,
50	"My Youth Romantic Comedy Is Wrong, as I
	Expected",
51	"2013",
52	"05-04-2013",
53	"Hachiman Hikigaya, siswa SMA yang sinis, dipaksa bergabung dengan klub relawan bersama Yukino Yukinoshita, di mana ia mulai menghadapi berbagai masalah dan perlahan mengubah pandangannya tentang kehidupan serta hubungan antarmanusia.",
54	"Hachiman Hikigaya, a cynical high school student, is forced to join a volunteer club where he works with Yukino Yukinoshita to help others, gradually challenging his pessimistic view of life and relationships.",
55	R.drawable.img_oregairu,
56	"https://myanimelist.net/anime/14813/Yahari_Ore_no_Seishun_Love_Comedy_wa_Machigatteiru?q=oregairu&cat=anime",
57	null)
58	)
59	}

Folder data: NetworkResult.kt

Tabel 4. Source Code NetworkResult.kt Soal 1

1	package com.example.modul5compose.data
2	
3	sealed class NetworkResult<out T> {
4	object Loading : NetworkResult<Nothing>()
5	data class Success<out T>(val data: T, val query: String = "") : NetworkResult<T>()
6	data class Error(val message: String) : NetworkResult<Nothing>()
7	}

Folder data - datastore: AppPreferencesRepository.kt*Tabel 5 Source Code AppPreferencesRepository.kt Soal 1*

1	package com.example.modul5compose.data.datastore
2	
3	import android.content.Context
4	import androidx.appcompat.app.AppCompatActivityDelegate
5	import androidx.core.os.LocaleListCompat
6	
7	object AppPreferencesRepository {
8	private const val PREF_NAME = "app_prefs"
9	private const val KEY_LANG = "lang"
10	
11	private fun getPref(ctx: Context) =
12	ctx.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE)
13	
14	fun saveLanguage(ctx: Context, lang: String) {
15	getPref(ctx).edit().putString(KEY_LANG,
16	lang).apply()
17	}
18	
19	fun applySavedLanguage(ctx: Context) {
20	val lang = getPref(ctx).getString(KEY_LANG, "id-
21	ID") ?: "id-ID"
	AppCompatActivity.setApplicationLocales(LocaleListCompat.forLanguageTags(lang))
	}
	}

Folder data - local - dao: AnimeDao.kt*Tabel 6. Source Code AnimeDao.kt Soal 1*

1	package com.example.modul5compose.data.local.dao
2	
3	import androidx.room.Dao
4	import androidx.room.Insert
5	import androidx.room.OnConflictStrategy
6	import androidx.room.Query
7	import
8	com.example.modul5compose.data.local.entity.AnimeEntity
9	import kotlinx.coroutines.flow.Flow
10	
11	@Dao
12	interface AnimeDao {
	@Query("SELECT * FROM anime_table ORDER BY id")

```

13     fun observeAll(): Flow<List<AnimeEntity>>
14
15     @Query("SELECT * FROM anime_table ORDER BY id")
16     suspend fun getAllOnce(): List<AnimeEntity>
17
18     @Insert(onConflict = OnConflictStrategy.REPLACE)
19     suspend fun insertAll(items: List<AnimeEntity>)
20
21     @Query("DELETE FROM anime_table")
22     suspend fun clearAll()
23 }

```

Folder data - local - database: AnimeDatabase.kt

Tabel 7. Source Code AnimeDatabase.kt Soal 1

```

1 package com.example.modul5compose.data.local.database
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7 import com.example.modul5compose.data.local.dao.AnimeDao
8 import
9     com.example.modul5compose.data.local.entity.AnimeEntity
10
11 @Database(
12     entities = [AnimeEntity::class],
13     version = 2,
14     exportSchema = false
15 )
16 abstract class AnimeDatabase : RoomDatabase() {
17     abstract fun animeDao(): AnimeDao
18
19     companion object {
20         @Volatile
21         private var instance: AnimeDatabase? = null
22
23         fun getInstance(context: Context): AnimeDatabase {
24             return instance ?: synchronized(this) {
25                 instance ?: Room.databaseBuilder(
26                     context.applicationContext,
27                     AnimeDatabase::class.java,
28                     "anime_database"
29                 )
30             }
31         }
32     }
33
34     .fallbackToDestructiveMigration(dropAllTables = true)

```

30	.build()
31	.also { instance = it }
32	}
33	}
34	}
35	}

Folder data - local - entity: AnimeEntity.kt

Tabel 8. Source Code AnimeEntity.kt Soal 1

1	package com.example.modul5compose.data.local.entity
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	import com.example.modul5compose.model.Anime
6	
7	@Entity(tableName = "anime_table")
8	data class AnimeEntity(
9	@PrimaryKey val id: Int,
10	val title: String,
11	val year: String,
12	val releaseDate: String,
13	val plotId: String,
14	val plotEn: String,
15	val imageRes: Int,
16	val url: String,
17	val posterUrl: String?,
18	val updatedAt: Long
19	)
20	
21	fun AnimeEntity.toDomain() = Anime(id, title, year,
22	releaseDate, plotId, plotEn, imageRes, url, posterUrl)
23	fun Anime.toEntity(ts: Long) = AnimeEntity(id, title, year,
	releaseDate, plotId, plotEn, imageRes, url, posterUrl, ts)

Folder data - remote: NetworkModule.kt

Tabel 9. Source Code NetworkModule.kt Soal 1

1	package com.example.modul5compose.data.remote
2	
3	import com.example.modul5compose.BuildConfig
4	import
	com.example.modul5compose.data.remote.api.TmdbApiService
5	import kotlinx.serialization.json.Json
6	import okhttp3.Interceptor

```

7 import okhttp3.MediaType.Companion.toMediaType
8 import okhttp3.OkHttpClient
9 import okhttp3.Request
10 import retrofit2.Retrofit
11 import
    com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
12
13 object NetworkModule {
14     private const val BASE_URL =
15         "https://api.themoviedb.org/3/"
16     private const val IMAGE_BASE_URL =
17         "https://image.tmdb.org/t/p/w500"
18
19     private val json = Json {
20         ignoreUnknownKeys = true
21     }
22
23     private val authInterceptor = Interceptor { chain ->
24         val request: Request = chain.request().newBuilder()
25         .addHeader("Authorization", "Bearer
26             ${BuildConfig.TMDB_READ_TOKEN}")
27         .addHeader("accept", "application/json")
28         .build()
29         chain.proceed(request)
30     }
31
32     private val client = OkHttpClient.Builder()
33     .addInterceptor(authInterceptor)
34     .build()
35
36     private val retrofit = Retrofit.Builder()
37     .baseUrl(BASE_URL)
38     .client(client)
39     .addConverterFactory(json.asConverterFactory("application/j
40         son".toMediaType()))
41     .build()
42
43     private val apiService: TmdbApiService by lazy {
44         retrofit.create(TmdbApiService::class.java)
45     }
46
47     fun provideApiService(): TmdbApiService = apiService
48
49     fun posterUrl(path: String?): String? {
50         if (path.isNullOrEmpty()) return null

```

47	return "\$IMAGE_BASE_URL\$path"
48	}
49	}

Folder data - remote - api: TmdbApiService.kt

Tabel 10. Source Code TmdbApiService.kt

1	package com.example.modul5compose.data.remote.api
2	
3	import
	com.example.modul5compose.data.remote.dto.TmdbSearchResponse
4	import retrofit2.http.GET
5	import retrofit2.http.Query
6	
7	interface TmdbApiService {
8	@GET("search/multi")
9	suspend fun searchTitle(
10	@Query("query") query: String,
11	@Query("language") language: String = "en-US",
12	@Query("include_adult") includeAdult: Boolean =
	false
13	): TmdbSearchResponse
14	}

Folder data - remote - dto: TmdbResponse.kt

Tabel 11. Source Code TmdbResponse.kt Soal 1

1	package com.example.modul5compose.data.remote.dto
2	
3	import kotlinx.serialization.SerialName
4	import kotlinx.serialization.Serializable
5	
6	@Serializable
7	data class TmdbSearchResponse(
8	val results: List<TmdbSearchResultDto> = emptyList()
9	)
10	
11	@Serializable
12	data class TmdbSearchResultDto(
13	val id: Int = 0,
14	@SerialName("title") val title: String? = null,
15	@SerialName("original_title") val originalTitle:
	String? = null,
16	@SerialName("name") val name: String? = null,

```

17     @SerializedName("original_name") val originalName: String?
    = null,
18     @SerializedName("overview") val overview: String? = null,
19     @SerializedName("poster_path") val posterPath: String? =
    null,
20     @SerializedName("release_date") val releaseDate: String? =
    null,
21     @SerializedName("first_air_date") val firstAirDate: String?
    = null,
22     @SerializedName("media_type") val mediaType: String? =
    null,
23     @SerializedName("genre_ids") val genreIds: List<Int> =
    emptyList(),
24     @SerializedName("original_language") val originalLanguage:
    String? = null,
25     @SerializedName("origin_country") val originCountry:
    List<String> = emptyList()
26 )

```

Folder data - repository: AnimeRepository.kt

Tabel 12. Source Code AnimeRepository.kt Soal 1

```

1 package com.example.modul5compose.data.repository
2
3 import com.example.modul5compose.data.AnimeData
4 import com.example.modul5compose.data.NetworkResult
5 import com.example.modul5compose.data.local.dao.AnimeDao
6 import com.example.modul5compose.data.local.entity.toDomain
7 import com.example.modul5compose.data.local.entity.toEntity
8 import com.example.modul5compose.data.remote.NetworkModule
9 import
    com.example.modul5compose.data.remote.api.TmdbApiService
10 import
    com.example.modul5compose.data.remote.dto.TmdbSearchResultD
    to
11 import com.example.modul5compose.model.Anime
12 import com.example.modul5compose.R
13 import kotlinx.coroutines.Dispatchers
14 import kotlinx.coroutines.flow.*
15
16 class AnimeRepository(
17     private val apiService: TmdbApiService,
18     private val animeDao: AnimeDao
19 ) {
20     private val seedMovies = AnimeData.animeList
21

```



```

22     fun getAnimeStream(lang: String, query: String = ""):
Flow<NetworkResult<List<Anime>>> = flow {
23         if (query.isBlank()) {
24             val cached = animeDao.getAllOnce().map {
it.toDomain() }
25             if (cached.isNotEmpty()) {
26                 emit(NetworkResult.Success(cached, ""))
27             } else {
28                 emit(NetworkResult.Loading)
29             }
30
31             try {
32                 val freshData = fetchRemoteData(lang)
33                 saveToCache(freshData)
34                 emit(NetworkResult.Success(freshData, ""))
35             } catch (e: Exception) {
36                 if (cached.isEmpty())
emit(NetworkResult.Error("Gagal memuat data. "))
37             }
38             } else {
39                 val results = performSearch(query, lang)
40                 emit(NetworkResult.Success(results, query))
41             }
42         }.flowOn(Dispatchers.IO)
43
44     private suspend fun performSearch(query: String, lang:
String): List<Anime> {
45         val apiLang = if (lang.startsWith("id")) "id-ID"
else "en-US"
46         val response = try { apiService.searchTitle(query,
apiLang) } catch (e: Exception) { null }
47
48         val usedIds = mutableSetOf<Int>()
49         return response?.results?.filter {
50             it.genreIds.contains(16) &&
(it.originalLanguage == "ja" ||
it.originCountry.contains("JP"))
51         }?.mapNotNull { dto ->
52             val seedMatch = seedMovies.find { seed ->
53                 val dtoTitle = (dto.title ?: dto.name ?:
"" ).lowercase()
54                 val seedTitle = seed.title.lowercase()
55                 dtoTitle.contains(seedTitle) ||
seedTitle.contains(dtoTitle)
56             }
57             val finalId = seedMatch?.id ?: dto.id

```

```

58         if (usedIds.contains(finalId))
return@mapNotNull null
59         usedIds.add(finalId)
60
61         mapDtoToAnime(dto, seedMatch, apiLang)
62     } ?: emptyList()
63 }
64
65     private suspend fun fetchRemoteData(lang: String):
List<Anime> {
66         val apiLang = if (lang.startsWith("id")) "id-ID"
else "en-US"
67         val currentCache =
animeDao.getAllOnce().associateBy { it.id }
68
69         return seedMovies.map { seed ->
70             val response = try {
apiService.searchTitle(seed.title, apiLang) } catch (e:
Exception) { null }
71             val remote =
response?.results?.pickBestMatch(seed)
72             if (remote != null) {
73                 mapDtoToAnime(remote, seed, apiLang)
74             } else {
75                 currentCache[seed.id]?.toDomain() ?: seed
76             }
77         }
78     }
79
80     private fun mapDtoToAnime(dto: TmdbSearchResultDto,
base: Anime?, apiLang: String): Anime {
81         val isIndo = apiLang == "id-ID"
82         val rawDate = dto.releaseDate ?: dto.firstAirDate
?: ""
83         val formattedDate = if (rawDate.contains("-")) {
84             val parts = rawDate.split("-")
85             if (parts.size == 3) "${parts[2]}-${parts[1]}-
${parts[0]}" else rawDate
86         } else rawDate
87
88         return Anime(
89             id = base?.id ?: dto.id,
90             title = if (isIndo) (dto.originalTitle ?:
dto.originalName ?: dto.title ?: "Unknown") else (dto.title
?: dto.name ?: dto.originalTitle ?: "Unknown"),
91             year = rawDate.take(4),

```

```

92         releaseDate = if (formattedDate.isNotBlank())
formattedDate else base?.releaseDate ?: "-",
93         plotId = if (isIndo) (dto.overview?.takeIf {
it.isNotBlank() } ?: base?.plotId ?: "") else (base?.plotId
?: ""),
94         plotEn = if (!isIndo) (dto.overview?.takeIf {
it.isNotBlank() } ?: base?.plotEn ?: "") else (base?.plotEn
?: ""),
95         posterUrl =
NetworkModule.posterUrl(dto.posterPath),
96         imageRes = base?.imageRes ?:
R.drawable.img_horimiya,
97         url = if (dto.title != null)
"https://www.themoviedb.org/movie/${dto.id}" else
"https://www.themoviedb.org/tv/${dto.id}"
98     )
99 }
100
101 private fun
List<TmdbSearchResultDto>.pickBestMatch(seed: Anime):
TmdbSearchResultDto? {
102     val normalizedSeed = seed.title.lowercase()
103     return firstOrNull {
104         val resTitle = (it.title ?: it.name ?:
"").lowercase()
105         val resYear = (it.releaseDate ?:
it.firstAirDate ?: "").take(4)
106         resYear == seed.year &&
(resTitle.contains(normalizedSeed) ||
normalizedSeed.contains(resTitle))
107     } ?: firstOrNull { (it.releaseDate ?:
it.firstAirDate ?: "").take(4) == seed.year } ?:
firstOrNull()
108 }
109
110 private suspend fun saveToCache(items: List<Anime>) {
111     animeDao.clearAll()
112     animeDao.insertAll(items.map {
it.toEntity(System.currentTimeMillis()) })
113 }
114 }

```

Folder model: Anime.kt*Tabel 13. Source Code Anime.kt Soal 1*

1	package com.example.modul5compose.model
2	
3	data class Anime(
4	val id: Int,
5	val title: String,
6	val year: String,
7	val releaseDate: String,
8	val plotId: String,
9	val plotEn: String,
10	val imageRes: Int,
11	val url: String,
12	val posterUrl: String?
13	)

Folder ui - navigation: AppNavigation.kt*Tabel 14. Source Code AppNavigation.kt Soal 1*

1	package com.example.modul5compose.ui.navigation
2	
3	import androidx.compose.runtime.Composable
4	import androidx.compose.ui.platform.LocalContext
5	import androidx.lifecycle.viewmodel.compose.viewModel
6	import androidx.navigation.compose.NavHost
7	import androidx.navigation.compose.composable
8	import androidx.navigation.compose.rememberNavController
9	import com.example.modul5compose.ui.screen.DetailScreen
10	import com.example.modul5compose.ui.screen.HomeScreen
11	import com.example.modul5compose.ui.screen.SettingsScreen
12	import com.example.modul5compose.viewmodel.AnimeViewModel
13	import
	com.example.modul5compose.viewmodel.AnimeViewModelFactory
14	
15	@Composable
16	fun AppNavigation() {
17	val nav = rememberNavController()
18	val vm: AnimeViewModel = viewModel(factory =
	AnimeViewModelFactory(LocalContext.current))
19	
20	NavHost(navController = nav, startDestination = "home")
	{
21	composable("home") { HomeScreen(nav, vm) }
22	composable("detail/{id}") { backStack ->

23	val id =
24	backStack.arguments?.getString("id")?.toIntOrNull()
25	DetailScreen(nav, vm, id)
26	}
27	composable("settings") { SettingsScreen(nav) }
28	}

Folder ui - screen: DetailScreen.kt

Tabel 15. Source Code DetailScreen.kt Soal 1

1	package com.example.modul5compose.ui.screen
2	
3	import androidx.compose.foundation.layout.*
4	import androidx.compose.foundation.rememberScrollState
5	import androidx.compose.foundation.verticalScroll
6	import androidx.compose.material.icons.Icons
7	import
8	androidx.compose.material.icons.automirrored.filled.ArrowBack
9	import androidx.compose.material3.*
10	import androidx.compose.runtime.*
11	import androidx.compose.ui.Alignment
12	import androidx.compose.ui.Modifier
13	import androidx.compose.ui.graphics.Color
14	import androidx.compose.ui.layout.ContentScale
15	import androidx.compose.ui.platform.LocalConfiguration
16	import androidx.compose.ui.text.font.FontWeight
17	import androidx.compose.ui.unit.dp
18	import androidx.navigation.NavController
19	import coil.compose.AsyncImage
20	import androidx.compose.ui.res.stringResource
21	import com.example.modul5compose.R
22	import com.example.modul5compose.viewmodel.AnimeViewModel
23	@OptIn(ExperimentalMaterial3Api::class)
24	@Composable
25	fun DetailScreen(navController: NavController, vm:
26	AnimeViewModel, id: Int?) {
27	val anime = id?.let { vm.animeById(it) }
28	val lang =
29	LocalConfiguration.current.locales[0].language
30	val fallbackOverview =
	stringResource(R.string.no_description_available)
	Scaffold(

```

31         topBar = {
32             CenterAlignedTopAppBar(
33                 title = { Text(anime?.title ?: "Detail",
fontWeight = FontWeight.Bold) },
34                 navigationIcon = {
35                     IconButton(onClick = {
navController.popBackStack() }) {
36
37                     Icon(Icons.AutoMirrored.Filled.ArrowBack, null)
38                 },
39                 colors =
TopAppBarDefaults.centerAlignedTopAppBarColors(
40                     containerColor = Color.Black,
41                     titleContentColor = Color.White,
42                     navigationIconContentColor =
Color.White
43                 )
44             )
45         }
46     ) { p ->
47         if (anime != null) {
48
49             Column(Modifier.padding(p).fillMaxSize().verticalScroll(rememberScrollState())) {
50                 AsyncImage(
51                     model = anime.posterUrl ?:
anime.imageRes,
52                     contentDescription = null,
53                     modifier =
Modifier.fillMaxWidth().height(400.dp),
54                     contentScale = ContentScale.Crop
55                 )
56                 Column(Modifier.padding(20.dp)) {
57                     Row(Modifier.fillMaxWidth(),
Arrangement.SpaceBetween, Alignment.CenterVertically) {
58                         Text(anime.title, style =
MaterialTheme.typography.headlineSmall, fontWeight =
FontWeight.Bold, modifier = Modifier.weight(1f))
59                         Text(anime.year, style =
MaterialTheme.typography.titleLarge, color = Color.Gray)
60                     }
61
62                     Text("${stringResource(R.string.label_release_date)}:
${anime.releaseDate}", Modifier.padding(vertical = 8.dp),
color = Color.Gray)

```

```

61 Text(stringResource(R.string.label_overview), style =
    MaterialTheme.typography.titleMedium, fontWeight =
    FontWeight.Bold, modifier = Modifier.padding(top = 16.dp))
62         val overview = (if
    (lang.startsWith("i")) anime.plotId else anime.plotEn)
63             .takeIf { it.isNotBlank() }
64             ?: fallbackOverview
65         Text(overview, Modifier.padding(top =
    8.dp))
66     }
67 }
68 } else {
69     Box(Modifier.fillMaxSize().padding(p),
    contentAlignment = Alignment.Center) {
70         Text("Data tidak ditemukan")
71     }
72 }
73 }
74 }

```

Folder ui - screen: HomeScreen.kt

Tabel 16. Source Code HomeScreen.kt Soal 1

```

1 package com.example.modul5compose.ui.screen
2
3 import android.content.Intent
4 import androidx.activity.compose.BackHandler
5 import androidx.compose.foundation.layout.*
6 import androidx.compose.foundation.lazy.*
7 import androidx.compose.foundation.shape.RoundedCornerShape
8 import androidx.compose.material.icons.Icons
9 import androidx.compose.material.icons.filled.*
10 import androidx.compose.material3.*
11 import androidx.compose.runtime.*
12 import androidx.compose.ui.Alignment
13 import androidx.compose.ui.Modifier
14 import androidx.compose.ui.graphics.Color
15 import androidx.compose.ui.layout.ContentScale
16 import androidx.compose.ui.platform.*
17 import androidx.compose.ui.res.stringResource
18 import androidx.compose.ui.text.font.FontWeight
19 import androidx.compose.ui.text.style.TextOverflow
20 import androidx.compose.ui.unit.dp
21 import androidx.compose.ui.unit.sp
22 import androidx.core.net.toUri

```

```

23 import androidx.navigation.NavController
24 import coil.compose.AsyncImage
25 import com.example.modul5compose.R
26 import com.example.modul5compose.data.NetworkResult
27 import com.example.modul5compose.model.Anime
28 import com.example.modul5compose.viewmodel.AnimeViewModel
29
30 @Composable
31 fun HomeScreen(navController: NavController, vm:
AnimeViewModel) {
32     val state by vm.animeState.collectAsState()
33     val query by vm.searchQuery.collectAsState()
34     val lang =
LocalConfiguration.current.locales[0].toLanguageTag()
35     val focus = LocalFocusManager.current
36     val listState = rememberLazyListState()
37
38     var lastData by remember {
mutableStateOf<List<Anime>>(emptyList()) }
39     var lastQ by remember { mutableStateOf("") }
40
41     if (state is NetworkResult.Success) {
42         val s = state as NetworkResult.Success
43         lastData = s.data
44         lastQ = s.query
45     }
46
47     LaunchedEffect(lastQ) {
48         if (lastQ.isEmpty() && lastData.isNotEmpty())
listState.scrollToItem(0)
49     }
50
51     LaunchedEffect(lang) { vm.loadAnime(lang) }
52
53     BackHandler(query.isNotEmpty()) {
54         vm.onSearchQueryChanged("", lang)
55         focus.clearFocus()
56     }
57
58     Scaffold(
59         topBar = {
60             Surface(color = Color.Black, shadowElevation =
4.dp) {
61                 Row(
62                     modifier =
Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).padding(horizontal = 16.dp),

```



```

63         verticalAlignment =
Alignment.CenterVertically
64     ) {
65         Text(stringResource(R.string.app_name),
style = MaterialTheme.typography.titleLarge, fontWeight =
FontWeight.Bold, modifier = Modifier.weight(1f))
66         IconButton(onClick = {
navController.navigate("settings") }) {
67             Icon(Icons.Default.Settings, null,
tint = Color.White)
68         }
69     }
70 }
71 }
72 ) { p ->
73     Column(Modifier.padding(p)) {
74         OutlinedTextField(
75             value = query,
76             onChange = {
vm.onSearchQueryChanged(it, lang) },
77             modifier =
Modifier.fillMaxWidth().padding(16.dp, 8.dp),
78             placeholder = {
Text(stringResource(R.string.search_placeholder)) },
79             leadingIcon = { Icon(Icons.Default.Search,
null) },
80             trailingIcon = {
81                 if (query.isNotEmpty()) {
82                     IconButton(onClick = {
vm.onSearchQueryChanged("", lang); focus.clearFocus() }) {
83                         Icon(Icons.Default.Close, null)
84                     }
85                 }
86             },
87             shape = RoundedCornerShape(12.dp),
88             singleLine = true
89         )
90
91         Box(Modifier.weight(1f).fillMaxSize()) {
92             if (lastData.isNotEmpty()) {
93                 MainList(lastData, navController,
lastQ.isNotEmpty(), listState)
94             }
95             if (state is NetworkResult.Loading &&
lastData.isEmpty()) {

```

```

96 CircularProgressIndicator(Modifier.align(Alignment.Center),
    color = Color.White)
97     }
98     if (state is NetworkResult.Error &&
    lastData.isEmpty()) {
99         Column(Modifier.fillMaxSize(),
    Arrangement.Center, Alignment.CenterHorizontally) {
100             Text((state as
    NetworkResult.Error).message)
101             Button(onClick = {
    vm.loadAnime(lang) }, Modifier.padding(top = 8.dp)) {
    Text("Coba Lagi") }
102         }
103     }
104 }
105 }
106 }
107 }
108
109 @Composable
110 fun MainList(data: List<Anime>, nav: NavController,
    isSearch: Boolean, state: LazyListState) {
111     val ctx = LocalContext.current
112     LazyColumn(state = state, modifier =
    Modifier.fillMaxSize()) {
113         if (!isSearch) {
114             item {
115
116                 Text(stringResource(R.string.title_highlight),
    Modifier.padding(16.dp), fontWeight = FontWeight.Bold)
117                 LazyRow(contentPadding =
    PaddingValues(horizontal = 16.dp), horizontalArrangement =
    Arrangement.spacedBy(10.dp)) {
118                     items(data.take(5)) { anime ->
119                         Card(onClick = {
    nav.navigate("detail/${anime.id}") }, Modifier.size(210.dp,
    115.dp)) {
120                             AsyncImage(anime.posterUrl ?:
    anime.imageRes, null, contentScale = ContentScale.Crop,
    modifier = Modifier.fillMaxSize())
121                         }
122                     }
123                 Spacer(Modifier.height(8.dp))
124             }
125         }

```

```

126         items(data, key = { it.id }) { anime ->
127             AnimeRow(anime, {
nav.navigate("detail/${anime.id}") }, {
ctx.startActivity(Intent(Intent.ACTION_VIEW,
anime.url.toUri())) })
128         }
129     }
130 }
131
132 @Composable
133 fun AnimeRow(item: Anime, onDetail: () -> Unit, onWeb: () -
> Unit) {
134     val isIndo =
LocalConfiguration.current.locales[0].language.startsWith("
i")
135     val description = (if (isIndo) item.plotId else
item.plotEn)
136         .takeIf { it.isNotBlank() }
137         ?:
stringResource(R.string.no_description_available)
138     Card(
139         Modifier.padding(16.dp, 8.dp).fillMaxWidth(),
140         colors = CardDefaults.cardColors(containerColor =
Color(0xFF1E1E1E)),
141         shape = RoundedCornerShape(12.dp)
142     ) {
143         Row(Modifier.padding(12.dp).height(145.dp)) {
144             AsyncImage(item.posterUrl ?: item.imageRes,
null, Modifier.width(95.dp).fillMaxHeight(), contentScale =
ContentScale.Crop)
145             Column(
146                 modifier = Modifier
147                     .padding(start = 12.dp)
148                     .fillMaxHeight()
149                     .weight(1f),
150                 verticalArrangement =
Arrangement.SpaceBetween
151             ) {
152                 Column {
153                     Text(item.title, fontWeight =
FontWeight.Bold, maxLines = 1, overflow =
TextOverflow.Ellipsis)
154                     Text("${stringResource(R.string.label_release_date)}:
${item.releaseDate}", style =
MaterialTheme.typography.labelSmall, color = Color.Gray)
155                     Text(

```

```

156         description,
157         style =
MaterialTheme.typography.bodySmall,
158         maxLines = 3,
159         overflow = TextOverflow.Ellipsis,
160         modifier = Modifier.padding(top =
4.dp)
161     )
162 }
163 Row(Modifier.fillMaxWidth(),
horizontalArrangement = Arrangement.End) {
164     TextButton(onWeb) {
Text(stringResource(R.string.btn_browser), fontSize =
12.sp) }
165     Button(onDetail) {
Text(stringResource(R.string.btn_detail), fontSize = 12.sp)
}
166     }
167 }
168 }
169 }
170 }

```

Folder ui - screen: SettingScreen.kt

Tabel 17. Source Code SettingScreen.kt Soal 1

```

1 package com.example.modul5compose.ui.screen
2
3 import androidx.activity.compose.LocalActivity
4 import androidx.compose.foundation.layout.*
5 import androidx.compose.material.icons.Icons
6 import
androidx.compose.material.icons.automirrored.filled.ArrowBa
ck
7 import androidx.compose.material3.*
8 import androidx.compose.runtime.Composable
9 import androidx.compose.ui.Alignment
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.graphics.Color
12 import androidx.compose.ui.res.stringResource
13 import androidx.compose.ui.text.font.FontWeight
14 import androidx.compose.ui.unit.dp
15 import androidx.navigation.NavController
16 import com.example.modul5compose.R

```

```

17 import
   com.example.modul5compose.data.datastore.AppPreferencesRepo
   sitory
18
19 @Composable
20 fun SettingsScreen(navController: NavController) {
21     val activity = LocalActivity.current
22
23     Scaffold(
24         topBar = {
25             Surface(color = Color.Black, shadowElevation =
26 4.dp) {
27                 Row(
28                     modifier =
29 Modifier.fillMaxWidth().statusBarsPadding().height(64.dp).p
30 adding(horizontal = 8.dp),
31                     verticalAlignment =
32 Alignment.CenterVertically
33 ) {
34                 IconButton(onClick = {
35 navController.popBackStack() }) {
36
37                 Icon(Icons.AutoMirrored.Filled.ArrowBack, null, tint =
38 Color.White)
39
40                 }
41
42                 Text(stringResource(R.string.title_settings), style =
43 MaterialTheme.typography.titleLarge, fontWeight =
44 FontWeight.Bold)
45
46                 }
47             }
48         ) { p ->
49             Column(Modifier.fillMaxSize().padding(p),
50 Arrangement.Center, Alignment.CenterHorizontally) {
51                 LangBtn(stringResource(R.string.btn_indonesia))
52 {
53                     activity?.let {
54
55 AppPreferencesRepository.saveLanguage(it, "id-ID")
56
57 AppPreferencesRepository.applySavedLanguage(it)
58                     it.recreate()
59
60                     }
61
62                 }
63                 Spacer(Modifier.height(16.dp))
64                 LangBtn(stringResource(R.string.btn_english)) {

```

```

48         activity?.let {
49
50     AppPreferencesRepository.saveLanguage(it, "en-US")
51
52     AppPreferencesRepository.applySavedLanguage(it)
53         it.recreate()
54     }
55 }
56
57
58 @Composable
59 fun LangBtn(text: String, onClick: () -> Unit) {
60     Button(
61         onClick = onClick,
62         modifier =
63     Modifier.fillMaxWidth().padding(horizontal = 48.dp),
64         colors = ButtonDefaults.buttonColors(containerColor
65     = Color(0xFF2196F3))
66     ) {
67         Text(text)
68     }
69 }

```

Folder viewmodel: AnimeViewModel.kt

Tabel 18. Source Code AnimeViewModel.kt Soal 1

```

1 package com.example.modul5compose.viewmodel
2
3 import androidx.lifecycle.ViewModel
4 import androidx.lifecycle.viewModelScope
5 import com.example.modul5compose.data.NetworkResult
6 import
7     com.example.modul5compose.data.repository.AnimeRepository
8 import com.example.modul5compose.model.Anime
9 import kotlinx.coroutines.Job
10 import kotlinx.coroutines.delay
11 import kotlinx.coroutines.flow.*
12 import kotlinx.coroutines.launch
13
14 class AnimeViewModel(private val repository:
15     AnimeRepository) : ViewModel() {
16     private val _state =
17     MutableStateFlow<NetworkResult<List<Anime>>>(NetworkResult.
18     Loading)
19 }

```

```

15     val animeState = _state.asStateFlow()
16
17     private val _query = MutableStateFlow("")
18     val searchQuery = _query.asStateFlow()
19
20     private var job: Job? = null
21
22     fun loadAnime(lang: String, q: String = _query.value) {
23         job?.cancel()
24         job = viewModelScope.launch {
25             repository.getAnimeStream(lang, q).collect {
26                 _state.value = it
27             }
28         }
29
30         fun onSearchQueryChanged(newQ: String, lang: String) {
31             _query.value = newQ
32             job?.cancel()
33             if (newQ.isBlank()) {
34                 loadAnime(lang, "")
35             } else {
36                 job = viewModelScope.launch {
37                     delay(500)
38                     loadAnime(lang, newQ)
39                 }
40             }
41
42             fun animeById(id: Int): Anime? {
43                 val s = _state.value
44                 return if (s is NetworkResult.Success) s.data.find
45                     { it.id == id } else null
46             }

```

Folder viewmodel: AnimeViewModelFactory.kt

Tabel 19. Source Code AnimeViewModelFactory.kt Soal 1

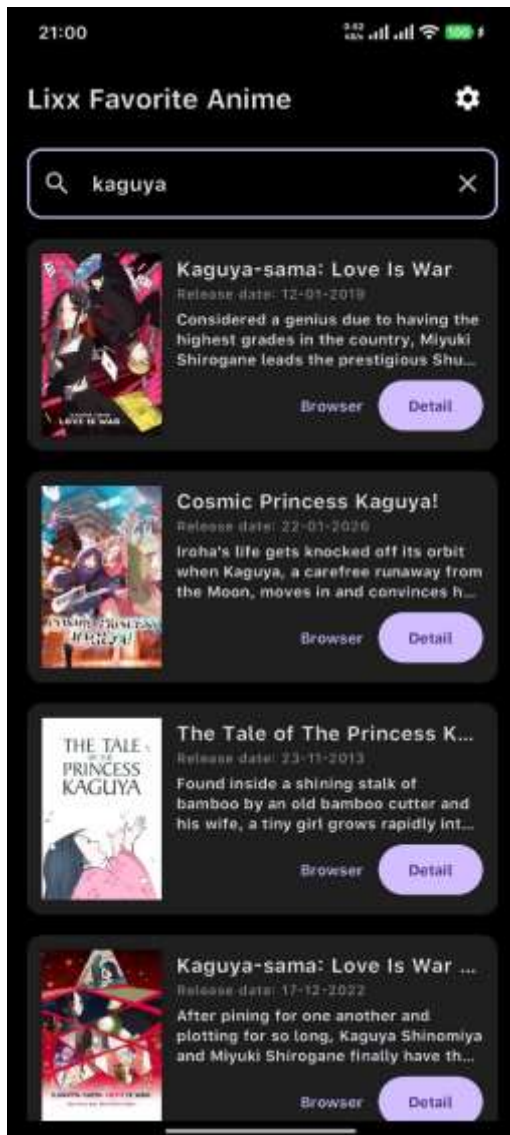
```

1 package com.example.modul5compose.viewmodel
2
3 import android.content.Context
4 import androidx.lifecycle.ViewModel
5 import androidx.lifecycle.ViewModelProvider
6 import
7     com.example.modul5compose.data.local.database.AnimeDatabase
8     import com.example.modul5compose.data.remote.NetworkModule

```

```
8 import
  com.example.modul5compose.data.repository.AnimeRepository
9
10 class AnimeViewModelFactory(private val context: Context) :
  ViewModelProvider.Factory {
11     override fun <T : ViewModel> create(modelClass:
  Class<T>): T {
12         if
  (modelClass.isAssignableFrom(AnimeViewModel::class.java)) {
13             val database =
  AnimeDatabase.getInstance(context)
14             val repository = AnimeRepository(
15                 apiService =
  NetworkModule.provideApiService(),
16                 animeDao = database.animeDao()
17             )
18             @Suppress("UNCHECKED_CAST")
19             return AnimeViewModel(repository) as T
20         }
21         throw IllegalArgumentException("Unknown ViewModel
  class")
22     }
23 }
```


B. Output Program



Gambar 1. Screenshot Hasil Tampilan List Jawaban Soal 1



Gambar 2. Screenshot Hasil Tampilan Detail Jawaban Soal 1

C. Pembahasan

MainActivity.kt

Kelas ‘MainActivity’ diturunkan dari ‘AppCompatActivity’, bukan dari ‘ComponentActivity’ biasa karena masih bergantung pada ‘AppCompatActivity’ untuk urusan pergantian bahasa. Di dalam ‘onCreate()’, baris pertama yang dieksekusi adalah ‘AppPreferencesRepository.applySavedLanguage(this)’, tujuannya supaya locale yang sudah disimpan sebelumnya langsung diterapkan sebelum UI apapun dirender. Kalau ini tidak dipanggil lebih awal, ada kemungkinan bahasa yang muncul di layar tidak sesuai dengan pilihan user sebelumnya.

Setelah itu, ‘setContent’ dipanggil untuk masuk ke dunia Compose. Di dalamnya langsung dibungkus dengan ‘MaterialTheme’ yang memakai ‘darkColorScheme’ dengan ‘background = Color.Black’, artinya skema warna gelap sudah ditentukan dari titik teratas ini. ‘Surface’ dengan ‘modifier = Modifier.fillMaxSize()’ dan ‘color = Color.Black’ dipakai sebagai wadah utama, lalu di dalamnya dipanggil ‘AppNavigation()’ yang mengurus semua perpindahan layar.

MyApplication.kt

Kelas ‘MyApplication’ diturunkan dari kelas ‘Application’ supaya bisa mengeksekusi sesuatu di level proses, bukan di level Activity. ‘onCreate()’ di-override, lalu di dalamnya dilakukan pengecekan apakah aplikasi sedang berjalan dalam mode debug melalui operasi bitwise ‘applicationInfo.flags and ApplicationInfo.FLAG_DEBUGGABLE’. Kalau hasilnya bukan nol, berarti sedang debug, dan ‘Timber.plant(Timber.DebugTree())’ dieksekusi. Timber adalah library logging, dan ‘DebugTree’ adalah implementasi bawaan yang secara otomatis menamai tag log berdasarkan nama kelas pemanggil. Dengan penempatan di sini, seluruh kode di aplikasi bisa pakai ‘Timber.d()’ atau ‘Timber.e()’ kapanpun, dan log hanya akan muncul saat build debug, tidak bocor ke build release.

Folder data: AnimeData.kt

File ini berisi object singleton ‘AnimeData’ yang menyimpan properti ‘animeList’, sebuah list statis dari model ‘Anime’. Isinya adalah lima data anime yang sudah di-hardcode langsung. Setiap item ‘Anime’ diisi dengan ‘id’, ‘title’, ‘year’, ‘releaseDate’, ‘plotId’

(deskripsi bahasa Indonesia), 'plotEn' (deskripsi bahasa Inggris), 'imageRes' yang merujuk ke drawable lokal, 'url' yang mengarah ke halaman MyAnimeList, dan 'posterUrl' yang diisi 'null' karena poster nanti diambil dari API TMDB. Data ini berfungsi sebagai seed data, yakni acuan awal saat melakukan pencarian ke TMDB agar hasil yang kembali bisa dipetakan ke data yang sudah dikenal aplikasi..

Folder data: NetworkResult.kt

'NetworkResult' adalah sealed class dengan tiga subclass: 'Loading', 'Success', dan 'Error'. 'Loading' dideklarasikan sebagai object karena tidak butuh data apapun, cukup representasi status saja. 'Success' adalah data class generik dengan parameter 'data: T' dan 'query: String' yang defaultnya string kosong, 'query' ini berguna untuk membedakan hasil pencarian dari hasil load biasa. 'Error' menyimpan 'message: String' untuk diteruskan ke UI saat terjadi kegagalan.

Folder data - datastore: AppPreferencesRepository.kt

Object singleton 'AppPreferencesRepository' mengurus penyimpanan dan penerapan preferensi bahasa menggunakan 'SharedPreferences'. Nama file preferensinya adalah "app_prefs" dan key yang dipakai adalah "lang". Fungsi 'getPref()' hanya sebagai helper privat yang mengembalikan instance 'SharedPreferences'. Fungsi 'saveLanguage()' menyimpan kode bahasa seperti "id-ID" atau "en-US" ke storage. Fungsi 'applySavedLanguage()' membaca nilai yang tersimpan, dengan default "id-ID" kalau belum pernah disimpan, lalu menerapkannya lewat 'AppCompatActivity.setApplicationLocales()' yang menerima 'LocaleListCompat.forLanguageTags()'.

Folder data - local - dao: AnimeDao.kt

'AnimeDao' adalah interface yang dianotasi '@Dao', dikenali Room sebagai Data Access Object. Ada empat fungsi di dalamnya. 'observeAll()' mengembalikan 'Flow<List<AnimeEntity>>' dengan query 'SELECT * FROM anime_table ORDER BY id', ini adalah fungsi reaktif yang otomatis emit ulang setiap ada perubahan di tabel. 'getAllOnce()' juga query yang sama tapi 'suspend fun' biasa yang hanya eksekusi sekali

tanpa observasi berkelanjutan. ‘insertAll()’ menerima list ‘AnimeEntity’ dan menggunakan ‘OnConflictStrategy.REPLACE’ artinya kalau ada data dengan id sama, yang lama akan diganti. ‘clearAll()’ menghapus seluruh isi tabel.

Folder data - local - database: AnimeDatabase.kt

Kelas ‘AnimeDatabase’ diturunkan dari ‘RoomDatabase’ dan dianotasi ‘@Database’ dengan parameter ‘entities = [AnimeEntity::class]’, ‘version = 2’, dan ‘exportSchema = false’. Properti abstrak ‘animeDao()’ adalah cara Room menghubungkan database ke DAO. Instance database dikelola dengan pola Singleton menggunakan ‘companion object’ dengan ‘@Volatile’ dan blok ‘synchronized’ untuk mencegah race condition saat diakses dari banyak coroutine sekaligus. ‘fallbackToDestructiveMigration(dropAllTables = true)’ digunakan supaya kalau versi database naik tapi tidak ada migration yang ditulis, tabel lama dihapus dan dibuat ulang dari awal.

Folder data - local - entity: AnimeEntity.kt

‘AnimeEntity’ adalah data class yang dianotasi ‘@Entity(tableName = “anime_table”)’. Setiap propertinya langsung menjadi kolom di tabel SQLite. ‘@PrimaryKey’ ada di ‘id: Int’. Properti ‘updatedAt: Long’ menyimpan timestamp Unix dalam milidetik, yang berguna kalau nanti ingin diimplementasikan logika cache expiry. Di bawah kelas ada dua extension function: ‘toDomain()’ untuk mengkonversi ‘AnimeEntity’ ke model ‘Anime’ yang dipakai di layer UI, dan ‘toEntity()’ yang menerima timestamp ‘ts: Long’ untuk mengkonversi balik dari ‘Anime’ ke ‘AnimeEntity’.

Folder data - remote: NetworkModule.kt

‘NetworkModule’ adalah object singleton yang membangun instance ‘Retrofit’ beserta seluruh dependensinya. ‘BASE_URL’ mengarah ke TMDB API versi 3, dan ‘IMAGE_BASE_URL’ adalah prefix URL poster ukuran ‘w500’. Instance ‘Json’ dikonfigurasi dengan ‘ignoreUnknownKeys = true’ supaya kalau response dari API mengandung field yang tidak ada di DTO, tidak langsung crash. ‘authInterceptor’ adalah ‘Interceptor’ yang menambahkan header ‘Authorization’ berisi Bearer token yang diambil dari ‘BuildConfig.TMDB_READ_TOKEN’, dan header ‘accept: application/json’. Token ini

disimpan di 'BuildConfig' supaya tidak ter-hardcode di source code. 'OkHttpClient' dibangun dengan interceptor tersebut, lalu 'Retrofit' menggunakannya bersama converter factory dari 'kotlinx.serialization'. Properti 'apiService' di-lazy supaya instance baru dibuat hanya saat pertama kali diakses. Fungsi 'posterUrl()' menggabungkan 'IMAGE_BASE_URL' dengan path dari API, dan mengembalikan null kalau path-nya kosong.

Folder data - remote - api: TmdbApiService.kt

'TmdbApiService' adalah interface Retrofit dengan satu fungsi 'searchTitle()'. Anotasi '@GET("search/multi")' menentukan endpoint yang dituju. Parameter 'query' dikirim sebagai query parameter dengan '@Query("query")', begitu juga 'language' dengan default "'en-US'" dan 'includeAdult' dengan default 'false'. Endpoint 'search/multi' dipilih karena bisa mengembalikan hasil dari berbagai jenis media (movie, tv, person) sekaligus, yang kemudian difilter di repository berdasarkan genre animasi.

Folder data - remote - dto: TmdbResponse.kt

'TmdbSearchResponse' adalah data class sederhana yang hanya berisi 'results: List<TmdbSearchResultDto>' dengan default list kosong. 'TmdbSearchResultDto' adalah representasi satu item dari hasil pencarian TMDB. Banyak propertinya bisa null karena endpoint 'search/multi' mengembalikan campuran tipe media yang masing-masing punya field berbeda. 'title' dan 'originalTitle' untuk tipe movie, sedangkan 'name' dan 'originalName' untuk tipe tv. 'posterPath' hanya berisi path relatif seperti "'/abc123.jpg'", bukan URL lengkap, sehingga butuh 'NetworkModule.posterUrl()' untuk dijadikan URL yang bisa dimuat Coil. Anotasi '@SerializedName' dipakai untuk memetakan nama field JSON yang pakai snake_case ke properti Kotlin yang pakai camelCase.

Folder data - repository: AnimeRepository.kt

'AnimeRepository' adalah kelas yang menerima 'TmdbApiService' dan 'AnimeDao' lewat constructor, ini adalah dependency injection manual tanpa framework. Fungsi utamanya adalah 'getAnimeStream()' yang mengembalikan 'Flow<NetworkResult<List<Anime>>>'. Kalau 'query' kosong, dia pertama cek cache di database; kalau cache ada, langsung emit 'Success' dengan data cache itu dulu sebelum fetch remote. Setelah itu tetap fetch remote

dengan `fetchRemoteData()` dan emit ulang hasilnya. Kalau cache kosong dan fetch remote gagal, emit `Error`. Kalau ada `query`, langsung panggil `performSearch()` tanpa sentuh cache sama sekali.

Fungsi `performSearch()` memanggil `apiService.searchTitle()` lalu memfilter hasilnya supaya hanya yang `genreIds.contains(16)` (genre Animasi di TMDB) dan bahasa aslinya Jepang atau negara asalnya Jepang yang lolos. Lalu setiap DTO dicari padanannya di `seedMovies` berdasarkan kemiripan judul. Kalau ketemu, id-nya dipakai dari seed supaya konsisten. Kalau tidak ketemu, id dari TMDB langsung dipakai. Set `usedIds` dipakai untuk mencegah duplikat. Fungsi `fetchRemoteData()` bekerja sebaliknya, iterasi setiap seed lalu cari satu per satu ke TMDB. Fungsi `pickBestMatch()` adalah extension function di list DTO yang mencoba mencocokkan berdasarkan tahun dan kemiripan judul dulu, kalau gagal fallback ke kecocokan tahun saja, kalau masih gagal ambil hasil pertama. Fungsi `mapDtoToAnime()` mengurus konversi DTO ke model domain, termasuk reformatting tanggal dari format TMDB `YYYY-MM-DD` ke `DD-MM-YYYY`, dan menentukan `url` apakah ke halaman movie atau tv TMDB berdasarkan ada tidaknya field `title`.

Folder model: Anime.kt

`Anime` adalah data class murni yang tidak memiliki anotasi apapun. Ini adalah model domain yang dipakai di layer ViewModel dan UI. Propertinya meliputi `id`, `title`, `year`, `releaseDate`, `plotId`, `plotEn`, `imageRes` (drawable resource integer), `url`, dan `posterUrl` yang nullable. Kelas ini sengaja dipisah dari `AnimeEntity` supaya kalau struktur database berubah, tidak langsung berdampak ke UI.

Folder ui - navigation: AppNavigation.kt

`AppNavigation()` adalah composable yang membangun seluruh grafik navigasi aplikasi. `rememberNavController()` dipakai untuk membuat dan menyimpan instance nav controller. `AnimeViewModel` dibuat sekali di sini menggunakan `viewModel(factory = AnimeViewModelFactory(LocalContext.current))` dan kemudian di-pass ke layar yang membutuhkannya. `NavHost` mendefinisikan tiga rute: `“home”` untuk `HomeScreen`, `“detail/{id}”` untuk `DetailScreen` yang menerima argumen id berupa string yang diparse ke `Int`, dan `“settings”` untuk `SettingsScreen`.

Folder ui - screen: DetailScreen.kt

‘DetailScreen’ menerima ‘navController’, ‘vm’, dan ‘id: Int?’. Data anime diambil dengan ‘vm.animeById(it)’ yang mencari di state ViewModel yang sedang aktif. Locale aktif dibaca dari ‘LocalConfiguration.current.locales[0].language’ untuk menentukan plot mana yang ditampilkan. ‘Scaffold’ dipakai dengan ‘CenterAlignedTopAppBar’ yang tombol backnya memanggil ‘navController.popBackStack()’. Konten utamanya dibungkus ‘Column’ dengan ‘verticalScroll’. ‘AsyncImage’ dari Coil memuat gambar dengan prioritas ke ‘posterUrl’, kalau null fallback ke ‘imageRes’. Penentuan overview yang ditampilkan memakai ‘takeIf { it.isNotBlank() }’ supaya kalau plot kosong, yang muncul adalah string dari resource ‘R.string.no_description_available’. Kalau ‘anime’ null, ditampilkan pesan “Data tidak ditemukan” di tengah layar.

Folder ui - screen: HomeScreen.kt

‘HomeScreen’ berisi beberapa bagian. State dari ViewModel diobservasi dengan ‘collectAsState()’. Variabel ‘lastData’ dan ‘lastQ’ dipakai untuk menyimpan data terakhir yang berhasil ditampilkan, supaya saat status ‘Loading’ muncul, list tidak tiba-tiba kosong. ‘LaunchedEffect(lang)’ memanggil ‘vm.loadAnime(lang)’ setiap kali locale berubah. ‘BackHandler’ aktif saat ‘query’ tidak kosong, fungsinya reset search dan clear focus keyboard. ‘OutlinedTextField’ adalah kolom pencarian dengan icon search di depan dan icon close di belakang yang muncul kondisional. Di dalam ‘Box’, ‘MainList’ ditampilkan kalau ‘lastData’ tidak kosong, ‘CircularProgressIndicator’ muncul hanya saat loading dan belum ada data sama sekali, dan pesan error plus tombol retry muncul saat error dan belum ada data.

Composable ‘MainList’ menggunakan ‘LazyColumn’ sebagai kontainer utama. Kalau bukan mode search, ada section highlight di atas berupa ‘LazyRow’ yang menampilkan lima anime pertama sebagai kartu gambar berukuran ‘210x115.dp’. Di bawahnya, semua item di-render lewat ‘AnimeRow’. ‘AnimeRow’ menampilkan satu item dengan layout ‘Row’ berisi gambar di kiri dan informasi di kanan. Plot yang ditampilkan dipilih berdasarkan locale aktif. Di bawah informasi ada dua tombol: ‘TextButton’ untuk buka URL di browser eksternal dengan ‘Intent.ACTION_VIEW’, dan ‘Button’ untuk navigasi ke detail.

Folder ui - screen: SettingScreen.kt

‘SettingsScreen’ mengambil referensi ke Activity yang sedang aktif lewat ‘LocalActivity.current’. Layoutnya berupa ‘Column’ di tengah layar berisi dua tombol yang dibuat dari composable ‘LangBtn’. Tombol pertama untuk bahasa Indonesia, tombol kedua untuk bahasa Inggris. Setiap klik memanggil tiga operasi berurutan: simpan bahasa ke SharedPreferences lewat ‘saveLanguage()’, terapkan locale baru lewat ‘applySavedLanguage()’, lalu panggil ‘it.recreate()’ untuk restart Activity supaya perubahan bahasa langsung terjadi. ‘LangBtn’ adalah composable helper yang menerima teks dan lambda onClick, dengan lebar penuh dan padding horizontal ‘48.dp’, warna biru ‘0xFF2196F3’.

Folder viewmodel: AnimeViewModel.kt

‘AnimeViewModel’ menyimpan dua ‘MutableStateFlow’: ‘_state’ untuk status data dan ‘_query’ untuk teks pencarian. Keduanya di-expose sebagai ‘StateFlow’ yang immutable ke luar. Variabel ‘job’ bertipe ‘Job?’ digunakan untuk menyimpan coroutine yang sedang berjalan, supaya bisa di-cancel sebelum menjalankan yang baru, mencegah race condition. Fungsi ‘loadAnime()’ untuk membatalkan tugas lama lalu launch baru yang mengumpulkan flow dari repository. Fungsi ‘onSearchQueryChanged()’ update nilai query, lalu kalau query kosong langsung load ulang tanpa delay, tapi kalau ada isi query diberi ‘delay(500)’ sebagai debounce supaya tidak menghantam API setiap ketikan. Fungsi ‘animeById()’ mencari anime di data yang sudah ada di state, hanya bekerja kalau state-nya ‘Success’.

Folder viewmodel: AnimeViewModelFactory.kt

‘AnimeViewModelFactory’ mengimplementasikan ‘ViewModelProvider.Factory’ supaya ‘AnimeViewModel’ bisa dibuat dengan dependensi kustom tanpa Hilt atau Dagger. Di dalam ‘create()’, instance ‘AnimeDatabase’ diambil lewat ‘getInstance(context)’, lalu ‘AnimeRepository’ dibuat dengan ‘apiService’ dari ‘NetworkModule.provideApiService()’ dan ‘animeDao’ dari database. Kalau kelas yang diminta bukan ‘AnimeViewModel’, dilempar ‘IllegalArgumentException’. Suppress “‘unchecked_cast’” dipasang karena cast ke generik ‘T’ memang tidak bisa diverifikasi di compile time.

Caching Strategy:

Strategi caching yang digunakan adalah Cache-First with Network Refresh. Cara kerjanya bisa dilihat di `'getAnimeStream()'` pada `'AnimeRepository.kt'`. Ketika fungsi ini dipanggil, data dari Room diambil terlebih dahulu lewat `'animeDao.getAllOnce()'` dan langsung di-emit ke UI. Setelah itu, request ke API tetap dijalankan di latar belakang untuk mengambil data terbaru, hasilnya disimpan ke Room lewat `'saveToCache()'` yang isinya memanggil `'animeDao.clearAll()'` lalu `'animeDao.insertAll()'` untuk mengganti data lama dengan yang baru, kemudian di-emit ulang ke UI sebagai pembaruan.

Alasan memilih strategi ini dipilih karena data anime seperti judul, deskripsi, dan poster bersifat semi-statis dan tidak berubah setiap saat, sehingga data cache dari sesi sebelumnya masih valid untuk langsung ditampilkan. Jika memakai Network-Only, pengguna harus menunggu response API setiap buka aplikasi dan layar akan kosong saat koneksi bermasalah. Jika memakai Cache-Only, data tidak pernah diperbarui dari sumbernya. Cache-First with Network Refresh menggabungkan keduanya, pengguna mendapat tampilan yang cepat dari cache sekaligus data yang selalu diperbarui secara otomatis di latar belakang, dan aplikasi tetap bisa menampilkan data meski sedang offline selama cache sudah pernah terisi sebelumnya.

TAUTAN GIT

Berikut adalah tautan untuk repositori GitHub yang telah dibuat

https://github.com/Fiezz65/MODUL-MOBILE/tree/master/MODUL_5/COMPOSE